UD1 · Parte 0 — Cápsula normativa: matemática discreta, lógica algorítmica y complejidad

Por qué existe esta cápsula

El RA1/CE1.a de **Sistemas de Big Data** menciona:

> conceptos básicos de matemática discreta, lógica algorítmica y complejidad computacional, y su aplicación para el tratamiento automático de la información por medio de sistemas computacionales.

En este módulo no vamos a estudiar matemática discreta como asignatura teórica. La trabajaremos sólo en lo que aporta valor para entender sistemas Big Data.

La base principal del curso será estadística aplicada, calidad de datos e interpretación de datos. Esta cápsula sirve para cubrir la parte normativa mínima de forma útil.

1. Matemática discreta útil en Big Data

La matemática discreta trabaja con elementos separados o contables.

En Big Data aparece en:

- conjuntos,
- relaciones,
- grafos,
- conteos,
- combinaciones,
- claves,
- identificadores,
- estructuras de datos.

Conjuntos

Un conjunto es una colección de elementos.

Se puede escribir así:

$$A = \{1, 2, 3, 4\}$$

Si `A` es el conjunto de clientes de una fuente y `B` el conjunto de clientes de otra fuente, las operaciones de conjuntos ayudan a integrar datos.

Ejemplo:

- conjunto de clientes,
- conjunto de productos,
- conjunto de eventos,
- conjunto de registros válidos,
- conjunto de registros duplicados.

Operaciones útiles:

Operación	Uso en datos
Unión	combinar fuentes
Intersección	encontrar coincidencias entre datasets
Diferencia	detectar registros que faltan en una fuente
Pertenencia	comprobar si un valor está permitido

Notación básica:

$A \cup B$ unión: elementos que están en A o en B

$A \cap B$ intersección: elementos que están en A y en B

$A - B$ diferencia: elementos que están en A pero no en B

Ejemplo con clientes:

```
clientes_crm = {'C001', 'C002', 'C003', 'C004'}
clientes_ventas = {'C003', 'C004', 'C005'}

todos = clientes_crm | clientes_ventas
comunes = clientes_crm & clientes_ventas
sin_ventas = clientes_crm - clientes_ventas

print(todos)       # {'C001', 'C002', 'C003', 'C004', 'C005'}
print(comunes)     # {'C003', 'C004'}
print(sin_ventas)  # {'C001', 'C002'}
```

Esto no es “matemática decorativa”: es exactamente la lógica que hay detrás de muchas tareas de integración y control de calidad.

Relaciones

Una relación conecta elementos.

Formalmente, una relación entre dos conjuntos A y B es un subconjunto del producto cartesiano:

$$R \subseteq A \times B$$

En datos, eso se parece mucho a una tabla con claves.

Ejemplos:

- cliente compra producto,
- usuario genera evento,
- pedido contiene líneas,
- ciudad pertenece a provincia.

Esto ayuda a entender datasets relacionales, joins y modelos de datos.

Ejemplo:

cliente_id	producto_id
C001	P10
C001	P20
C002	P10

Esta tabla representa la relación “cliente compra producto”. Un `join` usa claves comunes para combinar relaciones.

```
SELECT c.nombre, v.producto_id
FROM clientes c
JOIN ventas v ON c.cliente_id = v.cliente_id;
```

Si la clave `cliente_id` no es única donde debería serlo, el join puede duplicar filas y falsear métricas. Por eso matemática discreta y calidad de datos van juntas.

Grafos

Un grafo conecta nodos mediante aristas.

Se suele escribir como:

$$G = (V, E)$$

Donde:

- V es el conjunto de vértices o nodos,
- E es el conjunto de aristas o conexiones.

Ejemplos:

- usuarios conectados en una red social,
- rutas de transporte,
- dependencias entre tareas,
- relaciones entre entidades.

No hace falta profundizar en teoría de grafos, pero sí entender que muchos problemas Big Data son problemas de relaciones.

Ejemplo de grafo de rutas:

```
Cádiz -> Sevilla
Sevilla -> Córdoba
Cádiz -> Málaga
Málaga -> Granada
```

Preguntas posibles:

- ¿Qué ciudades están conectadas directamente?
- ¿Cuál es el camino más corto entre dos ciudades?
- ¿Qué nodo concentra más conexiones?
- ¿Qué ruta se rompe si falla una conexión?

En Big Data, estas preguntas aparecen en logística, redes sociales, ciberseguridad, recomendadores y análisis de dependencias.

2. Lógica algorítmica

La lógica algorítmica consiste en expresar reglas de decisión de forma clara.

En datos aparece constantemente:

- filtros,
- condiciones,
- validaciones,
- reglas de limpieza,
- transformaciones,
- pipelines.

Ejemplo:

```
SI edad < 0 ENTONCES dato inválido
SI email está vacío ENTONCES registro incompleto
SI precio_total != unidades * precio_unitario ENTONCES inconsistencia
```

En Big Data, estas reglas deben ser reproducibles y automatizables.

La misma idea en Python:

```python
df['precio_valido'] = df['precio_total'] == df['unidades'] * df['precio_unitario']
df['edad_valida'] = df['edad'].between(0, 120)
df['registro_valido'] = df['precio_valido'] & df['edad_valida']
```

Y la misma idea en SQL:

```sql
SELECT *
FROM ventas
WHERE precio_total <> unidades * precio_unitario;
```

Una regla lógica buena debe ser:

- explícita,
- medible,
- reproducible,
- documentada,
- revisable cuando cambia el negocio.

3. Complejidad computacional básica

La complejidad computacional estudia cómo crece el coste de un algoritmo cuando aumenta el tamaño de los datos.

No vamos a hacer teoría avanzada. Necesitamos entender una idea:

> Una solución que funciona con 1.000 filas puede no funcionar con 100 millones.

Órdenes de crecimiento básicos

Notación	Idea	Ejemplo intuitivo
O(1)	coste constante	acceder a un elemento por clave
O(n)	crece linealmente	recorrer todos los registros
O(n log n)	crece más que lineal	ordenar datos eficientemente
O(n²)	crece cuadráticamente	comparar todos contra todos

En Big Data, evitar O(n²) suele ser crítico.

Comparación numérica

Imagina tres algoritmos y tres tamaños de datos:

n	O(n)	O(n log₂ n)	O(n²)
1.000	1.000	~9.966	1.000.000
1.000.000	1.000.000	~19.931.569	1.000.000.000.000
100.000.000	100.000.000	~2.657.542.475	10.000.000.000.000.000

La diferencia no es académica. Un proceso $O(n^2)$ puede parecer aceptable en clase con 1.000 filas y ser inviable con datos reales.

Ejemplo típico de problema:

```python
# Mal enfoque para Big Data: compara cada fila con todas las demás.
for a in registros:
    for b in registros:
        comparar(a, b)
```

Mejor enfoque: usar claves, índices, particiones o agregaciones.

```python
# Mejor idea: agrupar por clave y comparar sólo candidatos relevantes.
por_cliente = df.groupby('cliente_id')
```

4. Relación con sistemas Big Data

Estas ideas ayudan a entender:

- por qué importan los índices,
- por qué se particionan datos,

- por qué se distribuye el procesamiento,
- por qué algunos joins son caros,
- por qué se usan formatos columnares como Parquet,
- por qué Spark reparte trabajo,
- por qué hay que medir coste y calidad.

Ejemplo de conexión directa:

Concepto matemático	Decisión técnica
Conjunto	detectar clientes que están en una fuente pero no en otra
Relación	diseñar joins y claves entre tablas
Grafo	analizar conexiones, rutas o dependencias
Lógica	escribir reglas de validación y limpieza
Complejidad	decidir si una transformación escala o no

5. Qué debes dominar

En esta cápsula no necesitas demostrar teoremas. Necesitas usar estas ideas para razonar sobre datos.

Al terminar, deberías poder:

- conjuntos aplicados a fuentes de datos,
- relaciones aplicadas a joins/modelado,
- grafos como idea para datos conectados,
- reglas lógicas de limpieza,
- complejidad básica para razonar sobre escalabilidad.

No hace falta que estudies:

- demostraciones formales,
- teoría profunda de grafos,
- álgebra discreta avanzada,
- cálculo de complejidad complejo,
- matemática desconectada de datos reales.

6. Idea clave

La idea clave es aplicar matemática mínima para tomar mejores decisiones técnicas:

- conjuntos para combinar datos,
- relaciones para modelar datos,
- lógica para limpiar y validar,
- complejidad para entender escalabilidad.

Lo importante es que puedas justificar decisiones técnicas en sistemas Big Data.

Si una decisión no se puede explicar con una regla, una métrica o una estimación de coste, probablemente todavía no se entiende lo suficiente.